

HANOI UNIVERSITY OF SCIENCE AND
TECHNOLOGY

School of Information and communications technology

DESIGN CONCEPTS REPORT

AIMS Project

Subject: ITSS Software Development

Group 18

Nguyễn Tuấn Đạt – 20235907

Thành Hữu Đạt – 20235909

Lê Tùng Lâm – 20235962

Chu Tuấn Linh – 20235963

Tạ Hải Luân – 20235969

Hanoi, May 2026

1. Coupling

1.1. Content Coupling

Related modules	Description	Improvement Direction
views.py and services.py services.py and models.py serializers.py and model relationships	No strong content coupling was found. Modules do not modify each other's private implementation or bypass public interfaces. views.py delegates product mutations to ProductService instead of directly editing product internals. The closest case is dynamic access to type-specific relations such as book_details, cd_details, dvd_details, and newspaper_details, but this is framework-level model access rather than unsafe content coupling.	Keep API views thin and continue routing all product mutations through ProductService. If more product types are added, move type-detail synchronization and serialization into a mapper/repository so relation-name knowledge does not spread.
QRPaymentScreen.tsx payments/api.ts backend VietQRService backend PaymentTransaction model	No strong content coupling was found in the VietQR payment flow. Frontend payment components do not directly access backend database tables or internal service state. The QR payment screen communicates with the backend only through public API endpoints for creating QR payment, checking payment status, and using mock callback during development. Backend payment logic remains inside VietQRService and PaymentService instead of being duplicated in the frontend.	Keep the API-first design. Frontend components should continue using payments/api.ts instead of calling backend endpoints directly from UI components. Payment verification, amount checking, callback validation, and order status updates must remain backend responsibilities.
PayPalPaymentView, PayPalCaptureView, PayPalRefundView and PaymentTransaction model	No strong content coupling was found. Payment views do not modify the transaction database fields directly or bypass public status methods; instead, they alter transaction states cleanly using explicit model methods like mark_completed() or mark_refunded().	Keep views thin and continue routing all transaction status changes through designated model functions or service layer abstractions.
CartService and Product stock data OrderPlacementService and Cart/Delivery/Invoice services Cart/Order views and services	No strong content coupling was found in the Place Order flow. Cart validation uses only public product availability data, and the order placement flow coordinates cart, delivery, and invoice modules through service boundaries instead	Keep cart, delivery, and invoice rules behind service interfaces. Do not put stock validation, delivery fee formulas, or invoice calculation directly into API views.

	of directly accessing internal implementation details.	
--	--	--

1.2. Common Coupling

Related modules	Description	Improvement Direction
services.py with TYPE_DETAIL_CONFIG and PRODUCT_FIELDS frontend/src/features/products/api.ts ProductTypeFields.tsx	Common coupling is low to moderate. There is no broad global mutable state. However, product subtype metadata is duplicated between frontend and backend, and X-Manager-Id: linh is hard-coded in frontend API calls. These are configuration coupling risks: if product subtype fields or manager identity handling change, multiple modules must be updated together.	Replace hard-coded manager id with authenticated user context after the Users/Auth business is implemented. Document product type fields in the API contract or expose a backend metadata endpoint for frontend form generation. Keep shared constants local to the product feature instead of adding broad project-level globals.
QRPaymentScreen.tsx and sessionStorage checkoutStorage.ts payments/api.ts and NEXT_PUBLIC_API_BASE_URL payments/types.ts and backend PaymentStatus	Common coupling is low to moderate. The main shared data source is sessionStorage, where invoice data is stored under the key aims.checkout.invoice before entering the payment step. The API base URL is shared through NEXT_PUBLIC_API_BASE_URL. Payment status values such as PENDING, SUCCESS, FAILED, and CANCELLED must be synchronized between frontend types and backend payment models.	Document the sessionStorage contract clearly with the Place Order owner so that order_id and final_payable_amount are always available before QR payment starts. Keep environment configuration outside source code. When new payment statuses or payment methods are added, update frontend types and backend models together.
payment/types/payment.ts and backend PaymentTransaction model payment/services/paymentApi.ts and PayPal gateway endpoints	Common coupling is low to moderate. The main shared data models are the payment status values and gateway enums that must remain tightly synchronized between the frontend TypeScript definitions and backend database models. If new payment states or routes change, both codebases require coordinated updates.	Maintain clear API contracts and coordinate updates to frontend payment types alongside backend model enums whenever changes occur. Keep environment configurations isolated.

checkoutStorage.ts and invoice/order data DeliveryFeeService constants Order/payment status values	Common coupling is low to moderate. Place Order shares checkout data such as order_id, invoice totals, delivery fee, and final_payable_amount with later payment steps. Delivery fee rules and order/payment status values also need to stay synchronized between backend contracts and frontend DTOs.	Document the checkout storage and invoice handoff contract. Keep delivery fee calculation as the backend source of truth and update frontend types whenever order or payment status values change.
--	--	--

1.3. Control Coupling

Related modules	Description	Improvement Direction
ProductService._sync_type_details() ProductService.update_product() ProductSelector.list_for_manager() ManagerProductsPage delete UI	Control coupling is moderate and intentional. Product type controls which subtype model is updated. Query flags such as include_deleted and search control selector behavior. In update flow, stock_quantity and stock_adjustment_reason together decide whether the action is a normal update or a stock adjustment. The frontend also disables delete when more than 10 products are selected, matching backend rules	Keep delete and stock rules enforced on the backend even if the frontend validates them too. Consider a dedicated stock-adjustment endpoint/service method if stock changes become a separate use case. Define query flags such as include_deleted in the API endpoint registry so callers do not infer behavior from implementation
QRPaymentScreen.tsx state machine handleCheckStatus() mockConfirmVietQRPayment() backend VietQRService.confirm_callback()	Control coupling is moderate and intentional. QRPaymentScreen uses screen state such as idle, creating, pending, checking, success, failed, and error to decide which UI and API behavior should run. User actions such as Check Payment Status trigger API calls, and backend responses determine whether the user stays on the QR page or is redirected to the success page. On the backend, provider_status controls whether a transaction is	Keep the state machine because payment flow needs clear step-by-step control. If the QR screen grows more complex, extract state transitions into a reducer or a smaller payment flow hook. Frontend should never force a successful state without backend confirmation.

	marked as SUCCESS or FAILED.	
PayPal payment views (PayPalPaymentView, PayPalCaptureView, PayPalRefundView) and PaymentService	Control coupling is moderate and intentional. The views evaluate response flags and incoming serializer payloads to control whether to invoke initiation, capture, or refund logic. On the backend, provider API outcomes directly control whether a transaction moves into a completed, failed, or refunded state.	Ensure core business orchestration remains strictly inside the service layer rather than creeping into view handlers. If control or validation logic expands, decouple via custom hooks or service factories.
OrderPlacementService checkout sequence DeliveryFeeService.calculate_delivery_fee() can_cancel_order() and determine_refund_action()	Control coupling is moderate and intentional. Checkout must follow a fixed sequence: validate cart stock, collect delivery information, calculate delivery fee, generate invoice, and hand off to payment. Delivery location and order value also control fee branches, while order status and payment method control cancellation/refund behavior.	Keep workflow control explicit but delegate detailed rules to focused services. Cancellation/refund decisions should remain backend-side so frontend screens do not infer business-critical state transitions.

1.4. Stamp Coupling

Related modules	Description	Improvement Direction
ProductService.create_product() ProductService.update_product() ProductForm.tsx ProductSerializer	Stamp coupling is moderate. Services receive broad dictionaries containing many product fields, and ProductForm submits a full ProductPayload during edit even when only a few fields changed. ProductSerializer receives whole Product objects and extracts subtype details from related models. This is practical for Coding 1 but can become fragile as the schema expands	Introduce explicit DTOs/dataclasses for create and update commands when the product workflow stabilizes. For update operations, send only changed fields from the frontend where possible. Consider separate serializers for common product fields and media subtype details
QRPaymentScreen.tsx and InvoiceCheckoutData checkoutStorage.ts /checkout/payment/qr/page.tsx	Stamp coupling is moderate. The QR payment screen may receive the full invoice data object, but it mainly needs only order_id and final_payable_amount to create a VietQR payment. Other invoice	Extract only the required fields before passing data to QRPaymentScreen. The parent page should convert the full invoice object into a minimal payment input DTO containing

	fields such as subtotal, VAT, and delivery fee are not always required by QRPaymentScreen. This is acceptable for the current integration stage but can make the component depend on more data than necessary.	order_id and final_payable_amount. This will reduce stamp coupling and make the QR payment component easier to reuse and test.
PayPalPaymentView, PayPalCaptureView, PayPalRefundView, serializers, and PaymentTransaction model	Stamp coupling is moderate. Payment API views update transaction records by directly invoking model instances and passing specific model methods, meaning the controllers know about the underlying ORM structure. While practical for the current stage, it creates dependency on the database schema.	Introduce an explicit repository layer (e.g., PaymentTransactionRepository) for handling state transitions if API views become too heavily bound to ORM details.
validate_cart_item_quantity() calculate_delivery_fee() validate_delivery_info() payment handoff data	Data coupling is the preferred form in Place Order. Several boundaries exchange only the required data: requested quantity and stock quantity for cart validation; province, weight, and order value for delivery fee; delivery fields for validation; and order id plus amount for payment handoff.	Preserve explicit parameter passing and small request/response DTOs. Avoid reading request, session, or global state deep inside checkout services.
Cart DTO and order creation DeliveryInfoForm and delivery validation Invoice object and payment handoff	Stamp coupling is moderate. Order placement may pass full cart, delivery, or invoice objects even when downstream modules only need selected fields such as product id, quantity, province, weight, order id, and final payable amount.	Introduce minimal DTOs for checkout commands, delivery information, invoice summary, and payment input. Pass only order_id and final_payable_amount to payment modules when possible.
ProductSelector.get_for_customer(): Product, ProductStatus, ProductDetailView	The view passes only product_id, while the selector returns one Product aggregate with subtype relations and excludes deleted products	Keep deleted-product visibility centralized here so customer detail behavior stays consistent with customer list behavior

1.5. Data Coupling

Related modules	Description	Improvement Direction
ProductHistoryService.log() ProductSelector.list_for_manager()	Data coupling is the best coupling form currently present.	Preserve explicit parameter passing for new Product

frontend/src/features/products/api.ts	Several module boundaries exchange only the data needed for the operation: product, action_type, performed_by, changes, reason, productId, search, and payload objects. Product pages/components do not know raw fetch details because HTTP calls are isolated inside feature API functions	Manager endpoints. Avoid reading request/session/global state deep inside services. Keep HTTP details inside features/products/api.ts rather than spreading fetch calls across UI components
payments/api.ts and apiClient.ts payments/api.ts and CreateVietQRPaymentRequest payments/api.ts and VietQRPaymentResponse QRPaymentScreen.tsx and payments/api.ts /checkout/success/page.tsx and getPaymentStatus()	Data coupling is the strongest and most preferred coupling type in the VietQR payment feature. API functions exchange only the required data through explicit DTOs. createVietQRPayment sends order_id and amount to the backend and receives a defined VietQR response containing transaction_id, status, amount, transaction_reference, qr_payload, and qr_image_url. The success page only needs transactionId to retrieve payment status.	Maintain explicit request and response DTOs for every payment operation. Keep HTTP communication inside payments/api.ts and avoid spreading fetch calls across UI components. When PayPal/Card payment is added, reuse the same pattern with separate request and response types.
PayPalGateway, PaymentService, RefundService, paymentApi.ts, and DTOs (PaymentResult, CaptureResult, RefundResult)	Data coupling is high and represents the cleanest boundary. The service layers interact with the payment gateways solely via interface abstractions and lightweight data transfer objects (PaymentResult, CaptureResult, RefundResult). No underlying HTTP clients or ORM details are leaked across boundaries.	Preserve the usage of strict, explicit request and response DTOs for all operations. Avoid spreading endpoint fetching details into presentation components by keeping them centralized within paymentApi.ts.
validate_cart_item_quantity() calculate_delivery_fee() validate_delivery_info() payment handoff data	Data coupling is the preferred form in Place Order. Several boundaries exchange only the required data: requested quantity and stock quantity for cart validation; province, weight, and order value for delivery fee; delivery fields for validation; and order id plus amount for payment handoff.	Preserve explicit parameter passing and small request/response DTOs. Avoid reading request, session, or global state deep inside checkout services.
ProductSelector.list_for_customer(): Product, ProductStatus,	The view passes primitive filter values (search, category,	Keep customer filtering in selector rather than API

ProductListCreateView	min_price, max_price, sort) and receives a Product queryset. The selector also depends on Product status to hide deleted products	views. If filters grow, introduce a small filter DTO to reduce parameter drift
-----------------------	---	--

2. Cohesion

2.1. Coincidental Cohesion

Related modules	Description	Improvement Direction
backend/apps/products/* frontend/src/features/products/* frontend/src/components/products/*	No major coincidental cohesion was found. The files are not random collections of unrelated functions; they are grouped around the Product Manager business area and all contribute to create, update, delete, list, or audit product behavior.	Avoid adding cart, payment, invoice, delivery, or notification logic into apps/products. Keep future manager-only utilities inside the product feature or a clearly named shared module
frontend/src/features/payments/* QRPaymentScreen.tsx PaymentStatus.tsx payments/api.ts	No coincidental cohesion was found. The payment files are not random collections of unrelated functions. They are grouped around the Pay Order by VietQR use case and all contribute to QR generation, payment status display, payment API communication, or checkout confirmation.	Avoid adding cart, product, delivery, or invoice calculation logic into the payments feature. Keep the payments feature focused only on payment method selection, QR payment display, status checking, and payment confirmation.
apps/payments (PayPal files), payment/paypal/page.tsx, PayPalPaymentButton.tsx, PaymentResult.tsx	No coincidental cohesion was found. The PayPal-related files are not arbitrary collections; they are strictly grouped around the 'Pay Order by Credit Card / PayPal' use case and contribute solely to checkout processing, button rendering, and transaction operations.	Avoid adding product catalogue, shopping cart, or delivery tracking logic into the payments area. Keep files focused purely on payment execution and outcome display.
backend/apps/carts/* backend/apps/orders/* CartScreen, DeliveryInfoForm, InvoiceSummary	No major coincidental cohesion was found for Place Order. Cart, delivery, and invoice modules are grouped around the customer checkout lifecycle rather than random unrelated responsibilities.	Avoid adding product management, payment gateway, user administration, or notification logic into carts/orders modules unless it belongs to the checkout boundary.

2.2. Logical Cohesion

Related modules	Description	Improvement Direction
ProductTypeFields.tsx ProductService._sync_type_details() ProductSerializer.get_type_details()	Logical cohesion exists where one module handles a family of similar operations selected by product type. ProductTypeFields renders Book/CD/DVD/Newspaper fields; ProductService synchronizes subtype detail models; ProductSerializer serializes subtype detail fields. This is acceptable because all branches are variants of product detail handling	Keep TYPE_DETAIL_CONFIG as the single backend mapping while subtype rules remain simple. Split subtype behavior into dedicated mapper/serializer classes when each product type gains complex validation or different workflows
PaymentStatus.tsx payments/types.ts /checkout/success/page.tsx	Logical cohesion is low and acceptable. PaymentStatus.tsx handles different payment statuses such as PENDING, SUCCESS, FAILED, and CANCELLED, but all branches are variations of the same responsibility: displaying payment status. The success page may support both VIETQR and PAYPAL in the future, but it still focuses on showing payment confirmation information.	Keep status rendering simple and centralized. If PayPal introduces different provider-specific status values, map them in the API or service layer before passing them to shared UI components.
PaymentGateway, PayPalGateway, payment/types/payment.ts	Logical cohesion exists and is acceptable. PaymentGateway acts as an abstract interface definition managing a family of payment actions (initiate, capture, refund), and types.ts organizes types related by domain. They handle multiple branches of logic, but all branches serve the single payment domain.	Keep gateway interfaces clean. If provider-specific authorization or token-caching logic grows complex, extract it into dedicated helper classes rather than expanding the core gateway.
DeliveryInfoValidator DeliveryFeeService Cancellation/refund decision helpers	Logical cohesion is acceptable because each module handles a family of related variants: delivery field rules, location-based fee branches, and status/payment-method	Keep validation, fee branching, and refund mapping centralized. Split into smaller validators or services only when each rule family becomes complex.

	refund decisions. These branches are variations of the same checkout responsibilities.	
--	--	--

2.3. Temporal Cohesion

Related modules	Description	Improvement Direction
ManagerProductsPage initial loading products/migrations/0001_initial.py	Temporal cohesion is low and not harmful. The product feature does not group unrelated tasks simply because they happen at startup. Page loading and database migration are normal framework lifecycle concerns	Do not put seed data, cache warmup, and unrelated setup work into product services. Keep migration files focused only on schema changes
/checkout/payment/qr/page.tsx initial loading QRPaymentScreen.tsx useEffect checkoutStorage.ts	Temporal cohesion is low and not harmful. Some actions happen during page initialization, such as reading invoice data from sessionStorage and starting QR payment creation. These actions are related to preparing the QR payment screen and are normal in a Next.js checkout page lifecycle.	Keep initialization logic limited to loading payment input and preparing the QR transaction. Do not add unrelated startup tasks such as cart clearing, notification sending, or order approval checks into the QR payment page.
payment/paypal/page.tsx initial loading (useEffect), apps/payments/migrations/0001_initial.py	Temporal cohesion is low and not harmful. Actions like fetching the user's current session or pulling token initializers happen together during the Next.js page initialization cycle. These tasks run at the same time because they prepare the user interface for input, which is standard behavior.	Limit page initialization logic to setting up the payment flow state. Do not bundle unrelated tasks, such as clearing shopping carts or triggering email notifications, directly into the page load sequence.
Cart review and stock check Delivery update and fee recalculation Invoice preparation before payment	Temporal cohesion is present but not harmful. Some actions happen together during checkout initialization or update events, such as checking stock before delivery, recalculating fee after delivery changes, and preparing	Keep initialization and recalculation tasks limited to checkout readiness. Do not add unrelated payment confirmation, email sending, or product management work into these steps.

	invoice before payment.	
--	-------------------------	--

2.4. Procedural Cohesion

Related modules	Description	Improvement Direction
ProductService.delete_products() ProductService.create_product() ProductService.update_product() ProductForm.handleSubmit()	Procedural cohesion is present in workflow methods. For example, delete_products() validates request limit, checks daily limit, loads products, decides status, saves changes, and writes history in a fixed order. This is reasonable for use-case orchestration because the steps must happen in order to preserve business rules	Keep one public service method per use case: create product, update product, delete products, and future stock-specific operations. Extract private helper methods only when they remove duplication or clarify business rules. Watch ProductService size so it does not become a large procedural script
QRPaymentScreen.handleCheckStatus() QRPaymentScreen mock success flow backend VietQRService.confirm_callback() /checkout/success/page.tsx	Procedural cohesion is present in the QR payment workflow. Checking payment status follows a fixed order: call the status API, receive the backend response, update UI state, and redirect to the success page if the status is SUCCESS. The backend callback flow also follows a required sequence: find the transaction, verify the amount, update transaction status, and update order status if available.	Keep procedural workflow methods small and readable. If the QR screen becomes too large, extract repeated sequences into helper functions or custom hooks. Business-critical steps such as amount verification and transaction confirmation should remain in backend services.
payment/paypal/page.tsx payment flow orchestration, PayPalCaptureView handler, RefundService execution	Procedural cohesion is present and controlled. The PayPal page coordinates an ordered sequence of actions: initializing the button, listening to the capture callback, and finally transitioning to render the result component. On the backend, refund execution validates the order state before interacting with the gateway.	Keep procedural UI flow methods small and readable. If the multi-step payment logic expands further, extract the state flow transitions into a dedicated custom hook (e.g., usePayPalCaptureFlow).
OrderPlacementService DeliveryInfoForm submit flow Cancellation/refund decision flow	Procedural cohesion is justified because Place Order is a multi-step business workflow.	Keep procedural methods small and readable. Continue delegating validation and

	The order of steps matters: validate cart, validate delivery information, calculate delivery fee, generate invoice, then proceed to payment or cancellation/refund decisions.	calculation details to CartService, DeliveryInfoValidator, DeliveryFeeService, and InvoiceService.
--	---	--

2.5. Communicational Cohesion

Related modules	Description	Improvement Direction
Product.clean() ProductSerializer and ProductPatchSerializer ProductTable.tsx	Communicational cohesion is high. These modules operate on the same product data structure and produce related outputs such as validation results, serialized API responses, or UI rows. This fits the Product Manager business because all operations revolve around product records	Keep product data validation close to Product and product serializers when it protects product consistency. Move cross-business rules out of Product when they start depending on order, cart, payment, or delivery state
QRPaymentScreen.tsx and PaymentStatus.tsx payments/api.ts and VietQRPaymentResponse /checkout/success/page.tsx and PaymentStatusResponse	Communicational cohesion is high and acceptable. These modules operate on the same payment transaction data and produce related outputs. QRPaymentScreen creates and displays a payment transaction, PaymentStatus renders the transaction status, and the success page fetches the same transaction information to display payment confirmation.	Keep payment transaction data as the central data object for the payment feature. Do not mix unrelated order management or product management data into payment components unless it is required for display. If success page display becomes larger, separate payment details and order details into smaller components.
PaymentTransaction, PaymentTransactionSerializer, paymentApi.ts	Communicational cohesion is high. These modules all consume or produce data derived from the same central PaymentTransaction data structures, transforming it into database records, standardized JSON payloads, or stateful frontend interfaces.	Keep transaction schemas gateway-agnostic so that it can continue to support multiple providers without breaking existing shared serialized endpoints.
CartService and CartScreen DeliveryInfoValidator and DeliveryInfoForm InvoiceService and InvoiceSummary	Communicational cohesion is high. These modules operate on shared checkout data structures: cart data, delivery information, and invoice totals.	Keep backend validation as the source of truth and frontend components focused on displaying or submitting the same DTOs. Separate payment details

	They produce related outputs such as stock messages, field-level errors, and invoice summaries.	from invoice display if the success page grows.
features/products/types.ts customer DTOs	The types describe product data exchanged between product API and product UI components	Keep unrelated cart/order/payment types out of this file

2.6. Functional Cohesion

Related modules	Description	Improvement Direction
ProductHistoryService.log() ProductSelector.list_for_manager() and get_for_manager() ProductService public methods ProductTable, ProductDeleteDialog, ProductTypeFields, product API functions	Functional cohesion is high in the best-isolated modules. ProductHistoryService.log() has one clear purpose: create normalized product history records. ProductSelector focuses only on product reads. ProductService public methods map directly to CUD business use cases. Frontend product components each serve a focused UI role	Maintain the current layering as more Product Manager features are added. Reduce ProductForm size later by extracting pricing, stock, dimension, and subtype field sections if the form grows. Keep feature API functions focused on HTTP communication only
payments/types.ts payments/api.ts QRPaymentScreen.tsx PaymentStatus.tsx /checkout/payment/page.tsx /checkout/payment/qr/page.tsx /checkout/success/page.tsx checkoutStorage.ts formatMoney.ts	Functional cohesion is high across the VietQR payment feature. payments/types.ts only defines payment-related DTOs and enums. payments/api.ts only handles payment API communication. QRPaymentScreen.tsx focuses on the QR payment user flow. PaymentStatus.tsx only displays payment status. checkoutStorage.ts only manages checkout session data, and formatMoney.ts only formats currency values.	Maintain the current separation of responsibilities as the payment feature expands. Add PayPal/Card-specific logic in separate components and API functions instead of mixing it into QRPaymentScreen. Keep shared utilities small and focused.
PayPalGateway, PaymentService, RefundService, PayPalPaymentButton, PaymentResult	Functional cohesion is exceptionally high across these modules. Each element performs exactly one distinct duty: PayPalGateway communicates with the	Maintain this precise separation of responsibilities as features scale. Ensure that UI elements remain pure and presentational, avoiding embedded data-fetching routines inside basic buttons

	provider API, RefundService handles refund rules, PayPalPaymentButton initiates requests, and PaymentResult acts as a pure presentation component.	or results panels.
CartService DeliveryInfoValidator DeliveryFeeService InvoiceService InvoiceSummary	Functional cohesion is strong in the Place Order modules. CartService focuses on cart readiness, DeliveryInfoValidator validates delivery inputs, DeliveryFeeService calculates shipping fee, InvoiceService calculates invoice totals, and InvoiceSummary renders invoice data only.	Maintain one clear reason to change for each module. Keep VAT and rounding policy centralized in InvoiceService and keep payment transaction state outside invoice rendering.